

Report Progetto WebServer

Emanuele De Francesco

June 2026

1 Introduzione

Il protocollo HTTP (HyperText Transfer Protocol) rappresenta il principale meccanismo di comunicazione utilizzato nel Web. Attraverso questo protocollo un client, come un browser o uno strumento da linea di comando come `curl`, può richiedere risorse ad un server remoto e riceverne il contenuto.

Un web server è un'applicazione che rimane in ascolto su una determinata porta di rete e risponde alle richieste provenienti dai client. Nella sua forma più semplice, un server web può limitarsi a distribuire file statici presenti sul file system, mentre implementazioni più avanzate possono fornire contenuti dinamici, autenticazione e numerosi altri servizi.

L'obiettivo di questo progetto è stato lo sviluppo di un server HTTP/1.0 in linguaggio C in grado di servire file statici presenti nella directory del progetto, gestire più connessioni contemporaneamente tramite la funzione `select()` e registrare le richieste ricevute attraverso un sistema di logging asincrono basato su thread.

2 Obiettivo del progetto

Il progetto richiede la realizzazione di un web server HTTP/1.0 capace di:

- accettare connessioni TCP provenienti da browser reali;
- interpretare richieste HTTP;
- supportare il metodo GET;
- servire file statici presenti nella directory del server;
- restituire codici di stato HTTP appropriati;
- gestire più client contemporaneamente;
- registrare gli accessi su file senza rallentare il server.

Durante lo sviluppo è stata data particolare attenzione alla modularità del codice e alla separazione delle responsabilità tra i vari componenti.

3 Analisi e progettazione

3.1 Approccio di sviluppo adottato

Lo sviluppo del server è stato effettuato in maniera incrementale.

Invece di implementare tutte le funzionalità contemporaneamente, il progetto è stato suddiviso in fasi successive. In una prima fase è stata realizzata la sola comunicazione TCP tra browser e server, verificando la corretta ricezione delle connessioni tramite socket.

Successivamente è stato implementato il parsing delle richieste HTTP, limitandosi inizialmente all'estrazione di metodo, path e versione del protocollo.

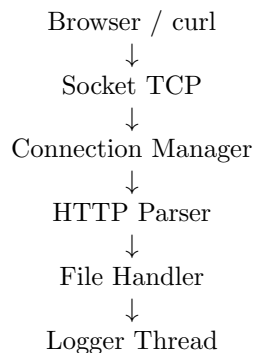
Una volta verificato il corretto funzionamento del parsing, è stato sviluppato il modulo di servizio dei file statici, che permette al server di individuare e inviare al client il file richiesto.

Terminata questa fase è stata implementata la gestione concorrente tramite `select()`, sostituendo il modello iterativo iniziale con un sistema in grado di monitorare simultaneamente più connessioni.

Infine è stato aggiunto il sistema di logging asincrono basato su thread e mutex. Questo approccio ha consentito di verificare ogni componente singolarmente prima di procedere alla fase successiva.

3.2 Architettura generale

Il flusso generale delle operazioni può essere rappresentato dal seguente schema:



Il server principale rimane costantemente in esecuzione all'interno di un ciclo infinito, monitorando nuove connessioni e client già connessi.

3.3 Struttura del progetto

Per migliorare leggibilità e manutenzione del codice, il progetto è stato suddiviso in moduli indipendenti.

File	Responsabilità
server.c	Punto di ingresso dell'applicazione e avvio del server.
create_socket.c	Creazione, configurazione e messa in ascolto della socket TCP.
connection_manager.c	Gestione delle connessioni concorrenti tramite la funzione <code>select()</code> .
client_handler.c	Ricezione delle richieste HTTP e coordinamento dei moduli successivi.
http-parser.c	Estrazione di metodo, path e versione dalla richiesta HTTP.
file_handler.c	Ricerca dei file richiesti, generazione delle risposte HTTP e invio del contenuto al client.
logger.c	Gestione del logging asincrono mediante thread dedicato e mutex.

4 Implementazione

4.1 Creazione della socket TCP

La comunicazione di rete è basata sul protocollo TCP/IP.

La socket viene creata mediante:

```
socket(AF_INET, SOCK_STREAM, 0);
```

Successivamente vengono configurati:

- famiglia di indirizzi IPv4;
- porta 8080;
- indirizzo locale del server.

La socket viene associata all'indirizzo tramite `bind()` e posta in ascolto mediante `listen()`.

L'accettazione delle connessioni avviene attraverso `accept()`.

4.2 Gestione delle connessioni

In una prima versione il server utilizzava un approccio iterativo, gestendo una sola connessione alla volta.

Successivamente è stato introdotto il modulo `connection_manager.c`, che utilizza la funzione `select()` per monitorare contemporaneamente il socket del server e tutte le connessioni attive.

Quando viene rilevata una nuova connessione, il file descriptor del client viene aggiunto all'insieme monitorato.

Quando un client invia dati, la richiesta viene delegata al modulo di gestione HTTP.

4.3 Parsing delle richieste HTTP

Le richieste vengono ricevute tramite `recv()` e salvate all'interno di un buffer. La prima riga della richiesta HTTP contiene tre informazioni fondamentali:

- metodo HTTP;
- path richiesto;
- versione del protocollo.

Ad esempio:

```
GET /index.html HTTP/1.1
```

Per memorizzare queste informazioni è stata definita una struttura:

```
typedef struct{
char method[16];
char path[256];
char version[16];
} HttpRequest;
```

L'estrazione dei dati viene effettuata tramite la funzione `sscanf()`.

4.4 Servizio dei file statici

Il modulo `file_handler.c` è responsabile della ricerca e dell'invio dei file richiesti.

Se il browser richiede:

```
/index.html
```

il server costruisce automaticamente il percorso locale del file:

```
./index.html
```

L'apertura viene effettuata mediante:

```
FILE *file = fopen(filepath, "r");
```

Se il file esiste viene restituito il codice:

```
HTTP/1.0 200 OK
```

e il contenuto viene inviato al client.

In caso contrario il server genera una risposta:

```
HTTP/1.0 404 Not Found
```

contenente una semplice pagina HTML di errore.

Per metodi diversi da GET viene restituito:

```
HTTP/1.0 405 Method Not Allowed
```

4.5 Gestione dei Content-Type

Per consentire al browser di interpretare correttamente il contenuto ricevuto, è stata implementata una funzione dedicata alla determinazione del content type. Attualmente il server supporta:

- text/html
- text/css
- application/javascript
- image/png
- image/jpeg

Il tipo di contenuto viene scelto dinamicamente in base all'estensione del file richiesto.

4.6 Gestione concorrente tramite select()

La funzione `select()` permette di monitorare contemporaneamente più socket. L'implementazione utilizza due strutture:

```
fd_set master_set;  
fd_set read_set;
```

Il socket del server e quelli dei client vengono inseriti nel set monitorato. Quando almeno una socket diventa pronta alla lettura, `select()` restituisce il controllo al programma permettendo di gestire immediatamente l'evento. È stata scelta questa soluzione invece di `epoll()` poiché il numero di connessioni previsto dal progetto non giustifica una maggiore complessità implementativa.

4.7 Logging asincrono con thread e mutex

Per evitare che le operazioni di scrittura su disco rallentino il server principale, è stato implementato un thread dedicato al logging.

Il sistema utilizza una coda circolare composta da:

```
log_queue[MAX_LOGS][MAX_MSG_LEN]
```

e due indici:

- head
- tail

La sincronizzazione tra thread avviene mediante:

```
pthread_mutex_t log_mutex;
```

Quando il server serve una richiesta, il thread principale inserisce una nuova entry nella coda.

Il thread logger preleva periodicamente le richieste dalla coda e le salva nel file:

`access.log`

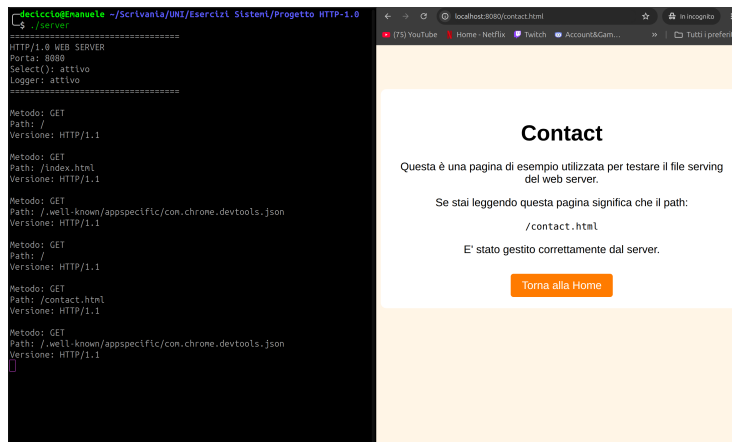
Questo approccio consente di separare la gestione delle richieste dalle operazioni di I/O su disco.

5 Test effettuati

5.1 Test tramite browser

Il server è stato testato utilizzando browser.
Sono stati verificati:

- caricamento della homepage;
- caricamento di più pagine HTML;
- gestione degli errori 404;
- caricamento di file CSS;
- caricamento di immagini PNG e JPG.



5.2 Test tramite curl

Per verificare il corretto funzionamento del protocollo HTTP sono stati effettuati diversi test tramite il comando:

```
curl http://localhost:8080/index.html
```

È stata inoltre verificata la gestione delle richieste verso file inesistenti.

```
decicciog@nauale ~/Scrivania/UNI/Esercizi Sistemi/Progetto HTTP-1.0
$ ./server
=====
HTTP/1.0 WEB SERVER
Porta: 8080
Select(): attivo
logger: attivo
=====

Metodo: GET
Path: /contact.html
Versione: HTTP/1.1
]

decicciog@nauale ~/Scrivania/UNI/Esercizi Sistemi/Progetto HTTP-1.0
$ curl http://localhost:8080/contact.html
<!DOCTYPE html>
<html>
<head>
<title>Contact</title>
<style>
body{
font-family: Arial, sans-serif;
background-color: #fff5e6;
text-align: center;
margin-top: 80px;
}
.container{
width: 600px;
margin: auto;
background: white;
padding: 30px;
border-radius: 10px;
}
a{
text-decoration: none;
color: white;
background: #ff7b00;
padding: 10px 20px;
border-radius: 5px;
}
</style>
</head>
<body>
<div class="container">
<h1> Contact</h1>
<p>
Questa è una pagina di esempio utilizzata per testare
il file server del web server.
```

5.3 Test con richieste concorrenti

Per verificare il corretto funzionamento della gestione concorrente è stato eseguito il seguente test:

```
for i in {1..100}; do
curl -s http://localhost:8080/index.html > /dev/null &
done

wait
```

Il server ha gestito correttamente tutte le richieste senza blocchi o errori.

```
decicciog@nauale ~/Scrivania/UNI/Esercizi Sistemi/Progetto HTTP-1.0
$ ./server
=====
HTTP/1.0 WEB SERVER
Porta: 8080
Select(): attivo
logger: attivo
=====

Metodo: GET
Path: /contact.html
Versione: HTTP/1.1

Metodo: GET
Path: /index.html
Versione: HTTP/1.1

Metodo: GET
Path: /index.html
Versione: HTTP/1.1

Metodo: GET
Path: /index.html
Versione: HTTP/1.1

Metodo: GET
Path: /index.html
Versione: HTTP/1.1

Metodo: GET
Path: /index.html
Versione: HTTP/1.1

Metodo: GET
Path: /index.html
Versione: HTTP/1.1

Metodo: GET
Path: /index.html
Versione: HTTP/1.1

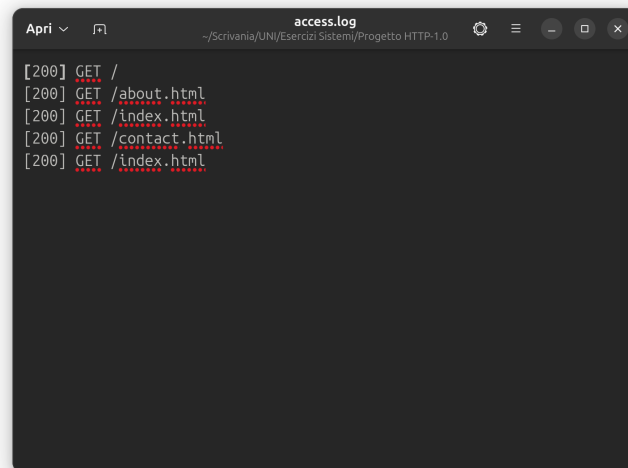
decicciog@nauale ~/Scrivania/UNI/Esercizi Sistemi/Progetto HTTP-1.0
$ for i in {1..100}; do
curl -s http://localhost:8080/index.html > /dev/null &
done

wait
[2] 26390
[3] 26391
[4] 26392
[5] 26393
[6] 26394
[7] 26395
[8] 26396
[9] 26397
[10] 26398
[11] 26399
[12] 26400
[13] 26401
[14] 26402
[3] 26391 done curl -s http://localhost:8080/index.html > /dev/
null
[4] 26392 done curl -s http://localhost:8080/index.html > /dev/
null
[3] 26403 done curl -s http://localhost:8080/index.html > /dev/
null
[2] 26404 done curl -s http://localhost:8080/index.html > /dev/
null
[5] 26393 done curl -s http://localhost:8080/index.html > /dev/
null
[4] 26405
[6] 26394 done curl -s http://localhost:8080/index.html > /dev/
null
[7] 26395 done curl -s http://localhost:8080/index.html > /dev/
null
[5] 26406
[6] 26407
[8] 26396 done curl -s http://localhost:8080/index.html > /dev/
null
[9] 26397 done curl -s http://localhost:8080/index.html > /dev/
null
```

5.4 Verifica del file di log

Durante i test è stato verificato il corretto funzionamento del sistema di logging. Le richieste vengono registrate nel formato:

```
[200] GET /index.html  
[404] GET /pagina.html
```



6 Considerazioni finali

6.1 Risultati ottenuti

Il progetto ha raggiunto tutti gli obiettivi prefissati.

Il server è in grado di:

- accettare connessioni TCP;
- interpretare richieste HTTP;
- servire file statici;
- gestire errori HTTP;
- supportare più client simultaneamente;
- registrare gli accessi tramite logger asincrono.

6.2 Limiti attuali

L'implementazione presenta alcune limitazioni:

- supporto esclusivo al metodo GET;
- assenza di contenuti dinamici;
- utilizzo di `select()` che non rappresenta la soluzione più scalabile.

6.3 Possibili sviluppi futuri

Tra le possibili evoluzioni del progetto:

- utilizzo di `epoll()`;
- gestione del metodo POST;
- implementazione di una cache dei file;
- supporto HTTPS.

7 Conclusioni

Il progetto ha permesso di approfondire concretamente i principali concetti della programmazione di rete in linguaggio C, tra cui socket TCP, parsing HTTP, gestione concorrente tramite multiplexing I/O e sincronizzazione tramite thread e mutex.

L'architettura modulare adottata ha semplificato lo sviluppo e la manutenzione del codice, consentendo di implementare progressivamente tutte le funzionalità richieste fino ad ottenere un web server HTTP/1.0 completo e funzionante.